

Table 2: A comparison of inodes and MFT

Feature	Inode (Linux/UNIX)	MFT (Windows/NTFS)
Purpose	Stores file/directory metadata and block location.	Database storing records for all files/directories and their metadata.
Identification	Inode number (i-number).	File ID.
Metadata storage	Stores permissions, ownership, timestamps, and block locations.	Stores permissions (through ACLs), time-stamps, attributes, file size, and data locations.
File name storage	Does not store file names; directories map names to inode numbers.	Stores file names within MFT records.
Small file handling	Data is stored in data blocks referenced by the inode.	Can store small files directly within the MFT record (resident data).
Hard links	Supports hard links, where multiple directory entries point to the same inode.	Supports hard links.
Architecture	Part of the design of UNIX-like filesystems.	A core database within the NTFS filesystem.
Timestamps	Access time (atime), modification time (mtime), change time (ctime).	Creation time, modification time, access time.

Copying creates a new file with a new inode, whereas moving updates only the directory information, leaving the inode untouched. If a file is moved to a different filesystem, it involves a copy operation, followed by a delete operation on the original file. Therefore, a new inode is assigned to the file in the destination filesystem.

Inode exhaustion

Inode exhaustion implies that a filesystem no longer has inodes left, even if the disk is not full. This may occur if several small files are created. To fix this, you can find and remove files that are not needed, or, if you can, make more inodes by reformatting the filesystem. Regular cleaning of temporary files, old log files, and unnecessary small files can help to prevent inode exhaustion. Command line tools, such as the `ncdu` (Ncurses Disk Usage Analyzer), based on the `du` command, but with an interactive and user-friendly interface, allow users to navigate through directories, sort them by size, and delete unnecessary directories and files directly from the interface.

Implications of inodes

Soft links: When a soft link is created, a new inode is used. Soft links can be created in different filesystems.

Hard links: When creating a hard link, the original file and all its hard links have the same inode number because hard links only provide name a new filename for the same data. The implementation of hard links is possible owing to the inodes. This is also the reason for the inability to create hard links across different filesystems, which may lead to conflicting inode numbers.

The inode's 'link count' field tracks the number of hard links pointing towards it. Deleting only one hard link decreases the link count. The file data is not deleted until all hard links are removed, and the link count reaches zero. Hard links remain valid even if the original file is renamed or moved within the same filesystem because they reference the inode directly rather than a path.

Windows version of inodes

Inodes are unique to UNIX-like filesystems. Windows OS NTFS (New Technology File System) uses a Master

File Table (MFT) to track the file details. MFT is a centralised structure that contains records of all files and folders in an NTFS volume.

The MFT combines an inode table and directory structure. Unlike inodes, the MFT stores some of the file's metadata directly within the MFT entry along with pointers to the data. This can make NTFS faster for small files; however, MFT can become fragmented over time. The MFT stores information about every file and folder in the NTFS volume. This includes size, timestamps, who can access it, and where the data is stored.

The contents of small files and directories (512 bytes or fewer) can be stored directly within the MFT entry. This reduces the need for additional disk reads, making NTFS faster than inode-based systems for accessing small files.

Implementation of inodes in newer filesystems

Dynamic inode allocation: Modern file systems, such as XFS, ZFS, btrfs, and APFS, have moved away from fixed-size inode tables. Instead, they use dynamic allocation methods that allow inodes to be created on demand. This eliminates the risk of running out of the inodes and